

Pemodelan Antrian: Sebuah Studi Kasus Pengubah Acak Diskrit

Catatan Kuliah Probabilitas dan Statistika

Pemodelan Antrian: Sebuah Studi Kasus Pengubah Acak Diskrit

Armein Z. R. Langi ¹, Meliana Christianti Johan ²

¹ Institut Teknologi Bandung armein@itb.ac.id

² Institut Teknologi Bandung meliana.christianti@it.maranatha.edu

Abstract

Dokumen ini dibuat untuk mahasiswa II-2111 Probabilitas dan Statistika. Kita lihat rantai distribusi probabilitas yang sangat penting dalam engineering. Hampir semua model sistem teknik (network, server, reliability, queueing) lahir dari rantai ini. Simulasi Poisson process sebenarnya sangat sederhana dan sangat bagus untuk membantu mahasiswa memahami konsep arrival acak dalam sistem.

Keywords: Antrian; Binomial; Poisson; Eksponensial; Python

Simulasi dulu baru teori. Seperti biasa kita mulai dengan impor python library yang diperlukan

```
import numpy as np
import matplotlib.pyplot as plt
```

Perhitungan Secara Manual

Katakanlah dalam suatu interval 60 menit rata rata 5 kendaraan per 10 menit pelanggan tiba di loket pelayanan, meskipun tidak bersamaan.

```
## parameter
lambda_rate = 5/10  ## rata-rata 5 request per
10 menit
T = 60  ## total waktu simulasi (menit)
```

Misalnya waktu pelayanan dengan no urut idx tiba di loket pelayanan adalah waktu_tiba[idx]. Durasi antar kedatangan pelanggan [idx] dengan idx-1 adalah selang_tiba[idx] dimana

waktu_tiba[idx] - waktu_tiba[idx-1] = selang_tiba[idx]

```
waktu_tiba = []
t = 0

while t < T:
    ## generate waktu antar kedatangan
    selang_tiba = np.random.exponential(1/
```

```
lambda_rate)
    t += selang_tiba

    if t < T:
        waktu_tiba.append(t)
```

Asumsi dalam sebuah interval waktu (misalnya 1 hari, atau 8 Jam, atau 1 jam) kedatangan terjadi secara acak

Contoh output:

```
tiba_total=len(waktu_tiba)

print("Arrival times:", waktu_tiba)
print("Total arrivals:", tiba_total)
```

Arrival	times:	
3.2000994089880783,		9.419685769673078,
12.49538745671005,		12.676694663362545,
14.982158748199897,		19.06415741966172,
20.063163161756272,		20.065154520374243,
24.749522432721157,		35.363900121657565,
39.771512314379706,		41.93152287771641,
43.05975847653915,		44.588057752177306,
45.39501430622771,		45.747532175489546,
49.934361679710356,		50.25344521940932,
52.74696550410564,		53.030029561009165,
57.89642786114467,		
57.95783375321613,		58.067594692478124,

```
58.60098228839955, 59.197631166161415]
Total arrivals: 26
```

Artinya: selama 60 detik terjadi 26 ** request**.

2. Visualisasi Arrival

Kemudian kita melihatnya secara visual

```
plt.eventplot(waktu_tiba)
plt.xlabel("Time")
plt.title("Poisson Arrival Process")
plt.show()
```

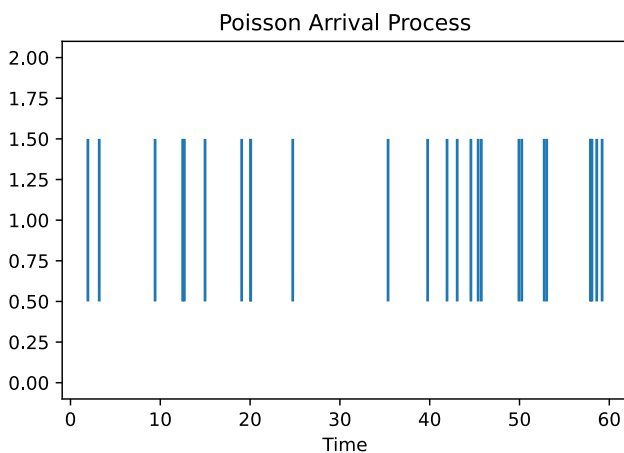


Figure 1: Grafik ini menunjukkan kedatangan request secara acak sepanjang waktu.

Kita langsung memahami bahwa:

arrival tidak teratur, tetapi rata-ratanya stabil

Simulasi Model Antrian

Simulasi **Poisson process** sebenarnya sangat sederhana dan sangat bagus untuk membantu mahasiswa memahami konsep **arrival acak** dalam sistem.

Kita akan mensimulasikan dua hal:

1. **jumlah kejadian dalam waktu tertentu** (Poisson)
2. **waktu antar kejadian** (Exponential)

Ini langsung menunjukkan hubungan teori yang kita bahas sebelumnya.

Seperti biasa kita mulai dengan impor python library yang diperlukan

```
import numpy as np
import matplotlib.pyplot as plt
```

1. Simulasi Poisson Process dengan Python

Ide dasar:

- waktu antar kedatangan mengikuti **Exponential**

- kita terus menambahkan waktu sampai melewati batas waktu total

Contoh: simulasi **request server selama 10 detik**.

Kita mulai dengan parameter dari random generator.

```
## parameter
lambda_rate = 2  ## rata-rata 2 request per detik
T = 10           ## total waktu simulasi (detik)
```

```
arrival_times = []
t = 0

while t < T:
    ## generate waktu antar kedatangan
    interarrival = np.random.exponential(1/
lambda_rate)
    t += interarrival

    if t < T:
        arrival_times.append(t)
```

Contoh output:

```
request=len(arrival_times)
print("Arrival times:", arrival_times)
print("Total arrivals:", request)
```

```
Arrival      times:      [0.24947022065891236,
0.705758691002432,      0.8157438412088059,
1.3920899732554146,      1.931987065974343,
2.7330129627650344,      3.0295950234436724,
3.138441961911517,      3.5984261542878575,
3.8220475049578475,      3.9798398255049254,
4.065220622468721,      4.161779684551403,
4.326349825267032,      4.4491237099303635,
6.196320929076668,      6.37392445573316,
6.600172693339279,      6.92238303687704,
7.3949852671, 8.530629222540139]
Total arrivals: 21
```

Artinya: selama 10 detik terjadi 21 ** request**.

2. Visualisasi Arrival

Kemudian kita melihatnya secara visual

```
plt.eventplot(arrival_times)
plt.xlabel("Time")
plt.title("Poisson Arrival Process")
plt.show()
```

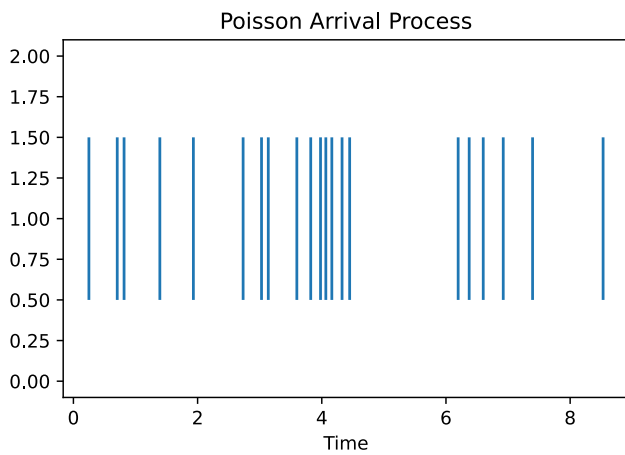


Figure 2: Grafik ini menunjukkan kedatangan request secara acak sepanjang waktu.

Kita langsung memahami bahwa:

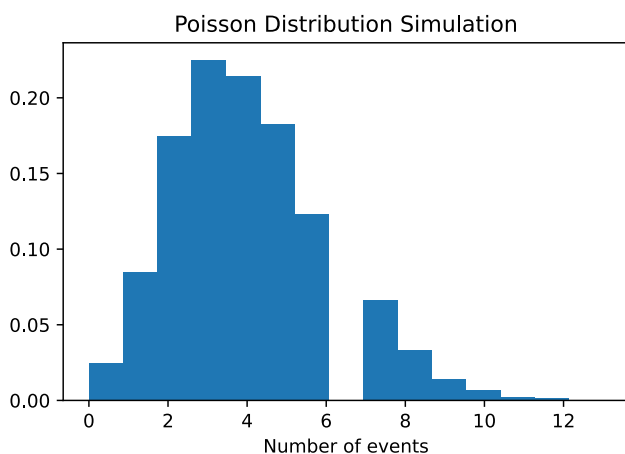
arrival tidak teratur, tetapi rata-ratanya stabil

3. Simulasi Distribusi Poisson

Sekarang kita simulasi **jumlah kejadian dalam interval waktu tetap**.

```
lambda_rate = 4    ## rata-rata 4 kejadian
samples = np.random.poisson(lambda_rate, 10000)

plt.hist(samples, bins=15, density=True)
plt.xlabel("Number of events")
plt.title("Poisson Distribution Simulation")
plt.show()
```



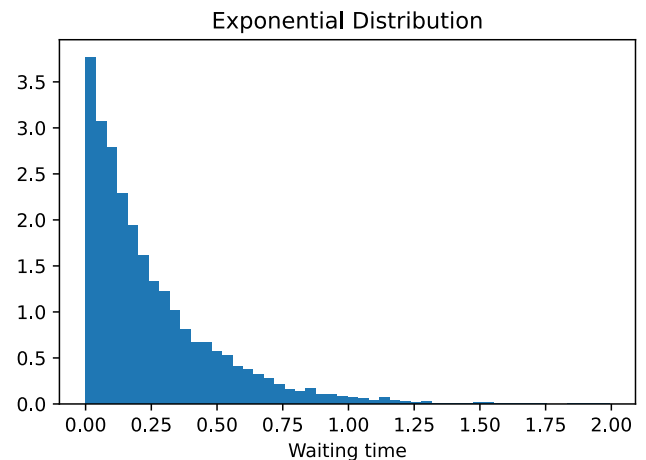
Ini akan menghasilkan histogram yang mengikuti distribusi **Poisson**.

4. Simulasi Waiting Time (Exponential)

Kita lihat distribusi waktu antar kedatangan.

```
interarrival = np.random.exponential(1/
lambda_rate, 10000)
```

```
plt.hist(interarrival, bins=50, density=True)
plt.xlabel("Waiting time")
plt.title("Exponential Distribution")
plt.show()
```



Mahasiswa akan melihat bahwa:

- banyak waktu tunggu **pendek**
- beberapa **panjang**

Ini ciri khas **exponential distribution**.

5. Demonstrasi yang sangat kuat di kelas

Anda bisa melakukan eksperimen sederhana:

Misalnya untuk **web server**:

```
lambda_rate = 5    ## 5 request per detik
T = 60             ## simulasi 1 menit
```

Lalu mahasiswa menghitung:

- total request
- waktu antar request
- distribusi arrival

Ini langsung menghubungkan teori:

Bernoulli → Binomial → Poisson → Exponential

ke **sistem nyata**.

6. Simulasi M/M/1 Queue (bonus kecil)

Contoh sangat sederhana:

```
arrival_rate = 2
service_rate = 3

n = 1000

interarrival = np.random.exponential(1/
```

```

arrival_rate, n)
service_time = np.random.exponential(1/
service_rate, n)

arrival_time = np.cumsum(interarrival)

start_service = np.zeros(n)
finish_service = np.zeros(n)

for i in range(n):
    if i == 0:
        start_service[i] = arrival_time[i]
    else:
        start_service[i] = max(arrival_time[i],
                                finish_service[i-1])

        finish_service[i] = start_service[i] +
                                service_time[i]

waiting_time = start_service - arrival_time

print("Average waiting time:",
      np.mean(waiting_time))

```

Average waiting time: 0.7018971085492183

Ini sudah menjadi **simulasi dasar sistem antrian**.

7. Insight penting untuk mahasiswa

Dengan simulasi ini mahasiswa melihat bahwa:

```

random events
→ Poisson arrival
→ Exponential waiting time
→ queueing systems

```

Ini membuat probabilitas terasa **hidup**, bukan sekadar rumus.

Rantai Distribusi Probabilitas dalam Engineering

```

Bernoulli → Binomial → Poisson → Exponential →
Queueing Theory

```

Menggunakan rujukan antara lain D. Mahayana and M. F. 'abbas [1], Kita bahas satu per satu secara intuitif.

1. Bernoulli — kejadian dasar (sukses / gagal)

$$P(X = 1) = p; P(X = 0) = 1 - p$$

Bernoulli adalah **atom dari probabilitas diskrit**.

Contoh:

- bit diterima benar / salah

- packet berhasil / hilang
- komponen hidup / gagal

Ini adalah **model kejadian tunggal**.

Dalam engineering digital:

```
error bit = Bernoulli trial
```

2. Binomial — jumlah sukses dalam n percobaan

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

Jika kita melakukan **banyak Bernoulli trial**, kita mendapat distribusi Binomial.

Contoh engineering:

- jumlah error dalam 1000 bit
- jumlah packet loss dalam 100 transmisi
- jumlah mahasiswa lulus ujian

Binomial menjawab:

```
berapa banyak kejadian sukses dalam n percobaan
```

3. Poisson — kejadian langka dalam waktu/ ruang

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

Jika:

- percobaan sangat banyak
- probabilitas sukses sangat kecil

maka Binomial menjadi **Poisson**.

Parameter:

$$\lambda = np$$

Contoh nyata engineering:

- jumlah request web per detik
- jumlah kendaraan lewat per menit
- jumlah error per MB data

Poisson menjawab:

```
berapa kejadian dalam interval waktu
```

4. Exponential — waktu antar kejadian

Jika kejadian mengikuti Poisson process, maka **jarak waktu antar kejadian** mengikuti distribusi **Exponential**.

$$f(t) = \lambda e^{-\lambda t}$$

Contoh:

- waktu antar telepon masuk
- waktu antar packet arrival
- waktu antar kegagalan komponen

Ini sangat penting karena memiliki sifat:

memoryless

Artinya:

masa depan **tidak bergantung masa lalu**.

5. Queueing Theory — sistem pelayanan

Jika:

- arrival mengikuti **Poisson**
- service time mengikuti **Exponential**

maka kita mendapatkan model antrian klasik:

M/M/1 queue

dimana

simbol	arti
M	Markov (Poisson arrival)
M	exponential service
1	satu server

Model ini dipakai untuk:

- server komputer
- call center
- jaringan internet
- sistem produksi

6. Diagram besar probabilitas sistem teknik

Struktur besar:

```
kejadian dasar      ↓ Bernoulli trial      ↓  
Binomial (jumlah sukses)  ↓ Poisson (kejadian  
per waktu)          ↓ Exponential (waktu antar  
kejadian)          ↓ Queueing systems
```

Ini adalah salah satu rantai konsep paling penting dalam engineering systems.

7. Intuisi besar (sangat penting)

Ada dua cara melihat dunia:

dunia statistik

menghitung kejadian

dunia sistem

kejadian → waktu antar kejadian → sistem antrian

Karena itu dalam engineering:

probability → stochastic process → queueing

8. Contoh nyata: server web

Misalnya:

- request datang **Poisson**
- server melayani **Exponential**

maka kita bisa menghitung:

- rata-rata antrian
- waktu tunggu user
- probabilitas server overload

Ini adalah dasar:

- cloud computing
- network engineering
- performance modeling

Terdapat juga bisa **hubungan yang lebih dalam lagi yang sangat elegan**, yaitu:

Bernoulli → Geometric → Negative Binomial →
Poisson → Exponential → Gamma

Ini sebenarnya adalah **keluarga distribusi yang berasal dari proses yang sama**.

Dan ini sangat indah secara matematis.

References

Bibliography

- [1] D. Mahayana and M. F. 'abbas, *Pengantar Probabilitas dan Statistika*. Bandung: Institut Teknologi Bandung Press, 2022, p. 292.